

an `<fx:Definition>` class with an `aname` property. The definition becomes an `ActionScript` class at compilation time. You can then declare instances of the reusable graphic element by referring to its name as an MXMLtag with the `fx` prefix. For example, in the code, the `<fx:Library>` tag is a direct child of `<s:Application>`. It contains this definition:

- `<fx:Definition name="CurvedArrow">`
- `<s:Path.../>`
- `</fx:Definition>`

The resulting `CurvedArrow` graphical element can then be used anywhere in the visual content of the document in which the `<fx:Library>` tag is declared.

Scaling graphic elements

An MXML graphic is rendered as a vector graphic; this means that its rendering is calculated mathematically rather than as a set of pixels (as is the case with bitmap graphics). As a result, you can increase or decrease the size of the graphic (a process known as scaling) without disturbing the graphic's resolution. When you do the same thing with a bitmap graphic, it's often pixelated, showing jagged edges. With vector graphics, the rendering stays clean and attractive.

The graphic that's defined in the `<fx:Library>` section is rendered three times, each time with a different value for the object's `scaleX` and `scaleY` properties:

- `<fx:CurvedArrow scaleX="1" scaleY="1"/>`
- `<fx:CurvedArrow scaleX="2" scaleY="2"/>`
- `<fx:CurvedArrow scaleX="4" scaleY="4"/>`



The result, with the three versions of the arrow graphic displayed side by side. Notice that the graphic's curves are smooth regardless of the scale. This is a benefit of vector graphics: because their presentation is calculated mathematically, they can adjust to whatever scale your application needs. A graphic element displayed in three scales